

Relatório de Implementação de Biblioteca de Grafos

Este relatório detalha a implementação de uma biblioteca de grafos em Java, a incorporação de algoritmos clássicos e o desenvolvimento de uma aplicação que a utiliza.

1. Estratégia de Representação de Grafos

Para a representação do grafo, foi utilizada a estratégia de **Listas de Adjacências**.

Nesta abordagem, cada vértice do grafo mantém uma lista de suas arestas de saída. Esta escolha oferece um bom equilíbrio entre a eficiência do uso de memória e o tempo de acesso aos vizinhos de um vértice.

A implementação é composta por três classes principais na biblioteca (lib): `Grafo<T>`, `Vertice<T>` e `Aresta<T>`.

- A classe `Grafo<T>` é o container principal. Ela armazena os vértices em um `Map<T, Vertice<T>>`, onde a chave é o dado genérico `T` do vértice e o valor é o próprio objeto `Vertice<T>`. Isso permite um acesso rápido a qualquer vértice a partir de seu dado.

```
// Trecho da classe Grafo.java
public class Grafo<T> {
    private Map<T, Vertice<T>> vertices;
    private boolean direcionado;

    public Grafo(boolean direcionado) {
        this.vertices = new HashMap<>();
        this.direcionado = direcionado;
    }

    public void adicionarVertice(T valor) {
        if (!vertices.containsKey(valor)) {
            vertices.put(valor, new Vertice<>(valor));
        }
    }
    // ... outros métodos
}
```

- A classe `Vertice<T>` armazena o dado genérico `T` e uma lista de objetos `Aresta<T>`, que representam as conexões que partem deste vértice.

```
// Trecho da classe Vertice.java
public class Vertice<T> {
    private T dado;
    private List<Aresta<T>> arestas;

    public Vertice(T dado) {
        this.dado = dado;
        this.arestas = new ArrayList<>();
    }

    public void adicionarAresta(Vertice<T> destino, float peso) {
        arestas.add(new Aresta<>(this, destino, peso));
    }
    // ... outros métodos
}
```

- Finalmente, a classe `Aresta<T>` conecta dois vértices (origem e destino) e armazena um peso do tipo `float`.

Adição de Aresta

- O método **adicionarAresta** na classe **Grafo** orquestra a criação de uma nova conexão. Ele recebe os dados de origem e destino, e o peso. Primeiro, verifica se ambos os vértices existem. Em caso afirmativo, recupera os objetos **Vertice** e invoca o método **adicionarAresta** do vértice de origem. Este, por sua vez, instancia um novo objeto **Aresta** e o adiciona à sua lista de adjacências. Se o grafo for não-direcionado, o processo é espelhado, adicionando uma aresta no sentido contrário.

```
// Trecho da classe Grafo.java
public boolean adicionarAresta(T origem, T destino, float peso) {
    if (!vertices.containsKey(origem) || !vertices.containsKey(destino)) {
        return false;
    }
    Vertice<T> vOrigem = vertices.get(origem);
    Vertice<T> vDestino = vertices.get(destino);
    vOrigem.adicionarAresta(vDestino, peso);

    if (!direcionado) {
        vDestino.adicionarAresta(vOrigem, peso);
    }
    return true;
}
```

Impressão do Grafo

Para visualizar a estrutura do grafo, o método **imprimirGrafo** foi implementado. Ele percorre a lista de valores do mapa de vértices. Para cada vértice, ele itera sobre sua lista de arestas e imprime cada uma. A representação textual da aresta é fornecida pelo método **toString()** da classe **Aresta**, resultando em uma listagem clara de todas as conexões e seus respectivos pesos.

```
// Trecho da classe Grafo.java
public void imprimirGrafo() {
    for (Vertice<T> v : vertices.values()) {
        for (Aresta<T> a : v.getArestas()) {
            System.out.println(a);
        }
    }
}
```

```
// Trecho da classe Aresta.java
@Override
public String toString() {
    return origem + " --(" + peso + "--> " + destino;
}
```

2. Algoritmos Clássicos Implementados

Foram implementados dois algoritmos clássicos: o **Algoritmo de Dijkstra** para encontrar o caminho mais curto em grafos direcionados e ponderados, e o **Algoritmo de Kruskal** para encontrar a Árvore Geradora Mínima (AGM) em grafos não direcionados e ponderados.

a. Algoritmo de Dijkstra

A implementação `Dijkstra.executar` calcula as distâncias mínimas de um vértice de origem para todos os outros vértices no grafo.

- **Implementação:** Utiliza-se uma Fila de Prioridade (`PriorityQueue`) para selecionar sempre o vértice com a menor distância ainda não finalizada, o que é a chave para a eficiência do algoritmo. Um `Map<T, Float> distancias` armazena as distâncias e um `Map<T, T> anteriores` reconstrói o caminho.

```
// Trecho da classe Dijkstra.java
public class Dijkstra {
    public static <T> void executar(Grafo<T> grafo, T origem) {
        // ... inicialização dos mapas

        PriorityQueue<T> fila = new PriorityQueue<>(Comparator.comparing(distancias::get));
        fila.add(origem);

        while (!fila.isEmpty()) {
            T atual = fila.poll();
            Vertice<T> verticeAtual = vertices.get(atual);

            for (Aresta<T> aresta : verticeAtual.getArestas()) {
                T vizinho = aresta.getDestino().getDado();
                float novaDist = distancias.get(atual) + aresta.getPeso();

                if (novaDist < distancias.get(vizinho)) {
                    distancias.put(vizinho, novaDist);
                    anteriores.put(vizinho, atual);
                    fila.add(vizinho);
                }
            }
        }
        // ... impressão dos resultados
    }
}
```

- **Análise de Complexidade:** A complexidade de tempo é $O(E \log V)$, onde V é o número de vértices e E é o número de arestas. A inserção e remoção na fila de prioridade levam tempo $O(\log V)$, e cada aresta é processada uma vez.

b. Algoritmo de Kruskal

A implementação `Kruskal.executar` constrói uma Árvore Geradora Mínima para um

grafo não direcionado.

- **Implementação:** O algoritmo primeiro ordena todas as arestas do grafo por peso em ordem crescente. Depois, itera sobre as arestas ordenadas, adicionando uma aresta à árvore resultante somente se ela não formar um ciclo com as arestas já adicionadas. Para detectar ciclos, utiliza-se a estrutura de dados Union-Find (representada aqui por um Map<T, T> pai e a função encontrarRaiz).

```
// Trecho da classe Kruskal.java
public class Kruskal {
    public static <T> void executar(Grafo<T> grafo) {
        // ...
        List<Aresta<T>> todasArestas = new ArrayList<>();
        // ... preenche a lista com todas as arestas

        todasArestas.sort(Comparator.comparing(Aresta::getPeso));
        Map<T, T> pai = new HashMap<>();
        // ... inicializa o union-find

        List<Aresta<T>> resultado = new ArrayList<>();

        for (Aresta<T> a : todasArestas) {
            T raiz1 = encontrarRaiz(pai, a.getOrigem().getDado());
            T raiz2 = encontrarRaiz(pai, a.getDestino().getDado());

            if (!raiz1.equals(raiz2)) {
                resultado.add(a);
                pai.put(raiz1, raiz2);
            }
        }
        // ... imprime a AGM
    }

    private static <T> T encontrarRaiz(Map<T, T> pai, T vertice) {
        // ... implementação da busca da raiz (Find)
    }
}
```

- **Análise de Complexidade:** A complexidade de tempo é dominada pela ordenação das arestas, resultando em **$O(E \log E)$** ou, de forma equivalente, **$O(E \log V)$** , já que E pode ser no máximo V^2 . As operações de Union-Find com otimizações (não totalmente implementadas aqui, mas em teoria) são quase constantes.

3. Aplicação Prática

O aplicativo AppGrafo.java resolve dois problemas práticos, dependendo da natureza do grafo fornecido pelo usuário:

1. **Para Grafos Direcionados (ex: mapa de rotas de sentido único):** O aplicativo calcula o **caminho mais curto** de um ponto de origem a todos os outros pontos, utilizando o **Algoritmo de Dijkstra**. Isso é útil para problemas de logística, roteamento de GPS, ou qualquer cenário onde se deseja encontrar o caminho de menor custo (distância, tempo, etc.).
2. **Para Grafos Não Direcionados (ex: rede de conexão de cidades):** O aplicativo calcula a **Árvore Geradora Mínima** utilizando o **Algoritmo de Kruskal**. Isso resolve o problema de conectar todos os pontos de uma rede (cidades, computadores, etc.) com o menor custo total possível (cabramento, estradas, etc.), garantindo que não haja redundâncias (ciclos).

Manual de Uso do Sistema

1. **Compilação:** Para obter os fontes, basta ter os arquivos .java no mesmo diretório ou em uma estrutura de pacotes (app e lib). Compile os arquivos usando o javac:
`javac app/AppGrafo.java lib/Grafo.java lib/Vertice.java lib/Aresta.java lib/Dijkstra.java lib/Kruskal.java`
2. **Execução:** Execute a classe principal AppGrafo:
`java app.AppGrafo`

3. Criação do Grafo:

Crie um arquivo de texto (ex: grafo.txt).

A **primeira linha** deve indicar o tipo do grafo: direcionado ou nao-direcionado.

As linhas seguintes definem o grafo:

- Para adicionar um vértice: NomeDoVertice
- Para adicionar uma aresta: Origem Destino Peso

Exemplo (grafo_direcionado.txt):

```
direcionado
A
B
C
D
A B 10
A C 5
B C 2
C D 1
```

Exemplo (grafo_nao_direcionado.txt):

nao-direcionado

Interação:

```
nao-direcionado
SP
RJ
BH
VT
SP RJ 400
SP BH 600
RJ BH 500
RJ VT 525
```

- Ao iniciar, o programa pedirá o caminho para o arquivo do grafo (ex: grafo_direcionado.txt).
- Após carregar, um menu será exibido com as opções:
 - **1 - Imprimir grafo:** Mostra todas as arestas do grafo.
 - **2 - Caminhamento em largura (BFS):** Percorre o grafo a partir de um vértice inicial.
 - **3 - Dijkstra ou Kruskal:** Executa o algoritmo apropriado para o tipo de grafo carregado (Dijkstra para direcionado, Kruskal para não direcionado).
 - **0 - Sair:** Encerra a aplicação.
- O usuário pode escolher as opções repetidamente até decidir sair.

4. Uso de Ferramentas de IA/LLM

Para a elaboração deste trabalho, foi utilizada a ferramenta **Gemini**, um Large Language Model (LLM) do Google, que atuou como assistente em diversas fases do projeto.

Processo de Utilização:

1. **Implementação da Biblioteca e Algoritmos:** A IA foi utilizada para gerar a estrutura inicial das classes Grafo<T>, Vertice<T>, e Aresta<T>. Para os algoritmos de Dijkstra e Kruskal, solicitamos implementações de referência que foram então estudadas, adaptadas para o nosso contexto genérico e integradas ao projeto.
2. **Criação do Aplicativo (AppGrafo.java):** A IA auxiliou na criação da lógica de leitura do arquivo de texto, sugerindo uma abordagem robusta para o parsing das linhas e o tratamento de exceções. Também foi consultada para estruturar o menu interativo do console e o loop principal da aplicação.
3. **Depuração:** Durante os testes, trechos de código que apresentavam erros ou comportamentos inesperados foram submetidos ao LLM para obter análises e sugestões de correção, o que agilizou o processo de depuração.
4. **Geração do Relatório:** Após a conclusão do código, o conjunto de arquivos .java foi fornecido à IA com a solicitação para gerar a base deste relatório técnico. O texto resultante foi então revisado, refinado e complementado.

Resultados Obtidos: O uso do LLM acelerou significativamente todo o ciclo de desenvolvimento, indo muito além da simples documentação. A ferramenta funcionou como um "programador em par" (pair programmer), oferecendo soluções de código, ajudando a sanar dúvidas conceituais e a identificar problemas, permitindo que o grupo focasse mais nos aspectos de alto nível do projeto do que em detalhes de implementação ou sintaxe.

App	Arthur Valentim
Lib	Bruno Alves, Caio Chiabai , Diego Rangel
Relatorio	Bruno Alves

Github: <https://github.com/CaioChiabai/TPA/tree/trab3>